



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Solving Chess Endgames

A Reinforcement Learning Approach

Valeria De Stasio, Christian Faccio, Giovanni Lucarelli

September 5, 2025

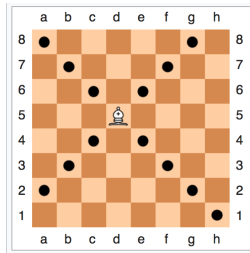
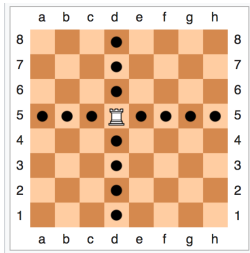
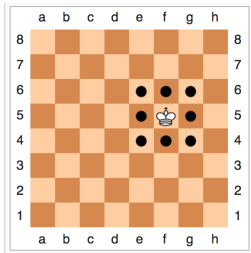
Objectives:

- Fastest checkmate against optimal player
- Experiment with training against random player
- Comparison of different RL approaches

Chess Programming Background

- 1890 El Ajedrecista: electro-mechanical KRK solver
- 1950 Claude Shannon: first article about chess programming
- 1997 *Deep Blue*: first computer program to beat the current world champion
- 2008 *Stockfish*: strong heuristic-based chess engine
- 2017 *Alphazero*: neural network-based approach that defeated Stockfish
- from 2018: more models developed based on *Alphazero*

Chess Rules

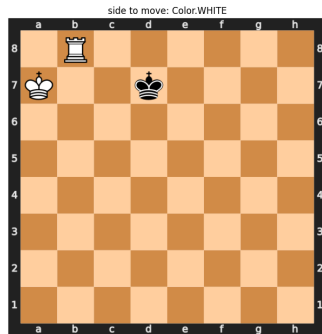


- **Win:** checkmate the opponent's king
- **Draw:** stalemate or insufficient material¹

¹threefold repetition and the fifty-move rule are not considered in our implementation

Elementary Endgames

- subset of endgames with few pieces left
- foundation for learning more complex endgames
- clear theoretical results: forced checkmate or draw
- Sygyzy endgame tablebases



Remark: white is always winning in elementary endgames!

MDP Formulation

Markov Game vs MDP

- Chess is a 2-player, turn-based, zero-sum game, formally a Markov Game
- Assume fixed black defender policy: **oracle** or **random**
- Black becomes part of the environment:
Markov Game $\rightarrow$ MDP

MDP: States

- State space $\mathcal{S}$: legal pieces positions and side-to-move

```
8/8/K7/8/8/5k2/7R/8 w - - 0 1
```

```
std::array<std::array<U64, 6>, 2> pieces;  
Color side_to_move;
```

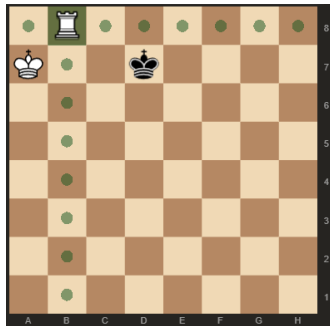
- Terminal state space $\bar{\mathcal{S}} \subset \mathcal{S}$: checkmate or draw positions (and stm). Defined procedurally

```
state.is_game_over() -> bool
```

MDP: Actions

- Action space $\mathcal{A}(s)$: set of legal moves for the current player.
- Defined procedurally:

```
state.legal_moves() ->  
std::vector<Move>
```



Transition probability $\Pr(S_{t+1} = s' | S_t = s, A_t = a)$

- **Oracle Defender:** deterministic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(oracle.reply()) // black deterministic
```

- **Random Defender:** stochastic evolution

```
state.do_move(Move)           // white deterministic
state.do_move(random_reply()) // black random
```

Goal: find the fastest mate

Reward:²

- checkmate: +1
- draw: -1 (always winning for white)
- encourage faster mates
 - step penalty: -2/-0.01
 - discount factor $\gamma = 0.99$

²Except for Value Iteration

RL Algorithms

Value Iteration: model-based approach

- KRk: 182.676 states
- Feasible number of states + perfect knowledge $\rightarrow$ Dynamic Programming
- $V(s) \leftarrow \max_a \sum_{s',r} p(s', r|s, a)[r(s, a, s') + \gamma V(s')]$
- $\pi(s) = \arg \max_a \sum_{s',r} p(s'|s, a)[r(s, a, s') + \gamma V(s')]$

Choice of parameters

- Step penalty: $-2 \rightarrow$ values = DTZ
- Draw penalty: $-100 \rightarrow$ slow mates $>$ quick draws
- $\gamma = 1$
- $\theta = 0.0001$

Q-Learning: model-free approach

- How good of a policy without knowing the “level” of the opponent?
- $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$
- $\pi(s) = \arg \max_a Q(s, a)$
- Off-policy, online and one-step.

Choice of parameters

- **Max steps** per episode: 50 $\rightarrow$ if it's too far from mate, agent may never reach terminal state
- Step penalty: -0.01
- Draw penalty: -1
- $\gamma = 0.99$
- Costant learning rate $\alpha = 0.05 \rightarrow$ smaller variance
- ϵ -greedy policy with $\epsilon = 0.3$ with decay

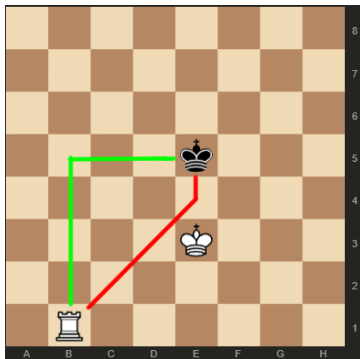
Actor-critic: approximate solution approach

- Non-elementary chess endgames → **curse of dimensionality**
- Use one-step actor-critic
- Update rules:
 - $\delta = r + \gamma V(s'; w) - V(s; w)$
 - $\theta \leftarrow \theta + \alpha^\theta \delta \nabla_\theta \log \pi(a|s; \theta)$
 - $w \leftarrow w + \alpha^w \delta \nabla_w V(s; w)$

Parameters Choice

- Policy: MLP(128,128,64), ReLU activations, softmax output
- Value function: $V(s) = \sum_{i=1}^{16} w_i \phi_i(s)$
- parameters:

Feature engineering



BK Manhattan distance
from side: maximum

BK Manhattan distance
from WR: ----

BK Chebyshev distance
from WR: ----

Opposition: False

Same rank: False

Same file: True

Monte Carlo Tree Search

- One of the first methods to solve chess
- No need to train!
- Heuristic method

Results

Assessment Metrics Overview

* DTM, DTZ etc * success, top1 etc

Why no convergence?

- Difficulties:
 - too many possible actions at each step
 - ideal opponent that never makes mistakes
 - very sparse positive rewards in training episodes
- Possible solutions:
 - more training time
 - implement draw by repetition and 50-move rule → more frequent terminal states

Policy interpretability through human chess principles

Video Animation of optimal ply vs our policy

maybe for a mate in 5 or more

Future Works

- * zobrist hashing and invariance properties in chess endgames to reduce state space cardinality

Why is it so difficult to solve chess endgames?

A player can sometimes afford the luxury of an inaccurate move, or even a definite error, in the opening or middlegame without necessarily obtaining a lost position. In the endgame ... an error can be decisive, and we are rarely presented with a second chance.

- Paul Keres

Thank You!

Why $\gamma = 0.99$?

The longest rkr endgame requires 16 white moves, hence $\gamma^{16} \approx 0.85$ is a good reward for reaching the goal and producing fast mates.